

Deep Dive — Operating Model, Oversight, Vendors, Change & Incident Practice

Part 12 —  AI Governance · 12-ai-governance/notes/02-operating-model-and-practice.md

This note answers the question the previous one deliberately left open: *who does the work, and what does "doing it" look like day to day?* A framework tells you what to think about. An operating model tells you which named human is on the hook for it.

1. The failure mode this note exists to prevent

Ask any large company "who owns AI risk here" and the honest answer is often "the AI ethics principles PDF from 2023, which nobody has re-read." That is not a governance program — it's a document. A real program has: a group that meets on a schedule and can say no, a function that assesses risk before deployment (not after a complaint), a person whose job title says "responsible AI," and — the part that's easiest to skip and most important — **a named executive who owns the outcome when the system is wrong**, not a committee, not "the model," not "AI."

2. The operating model: who does what

Role	Borrows from	Does	Does NOT do
AI governance committee / council	Corporate risk committees	Sets policy, approves the risk-tiering rubric, reviews High/Critical-tier use cases before launch, owns the AI use-case inventory	Doesn't review every prompt change — that's too slow and the wrong altitude
Model risk management (MRM) function	Banking/financial -services MRM (SR 11-7 lineage)	Independently validates a <i>specific</i> model/system: does it perform as claimed, is it robust, does the eval suite actually cover the failure modes	Doesn't build the model — independence from the build team is the entire point
Responsible AI office	Product/policy trust & safety teams	Owns the standards (model card template, oversight design guidance, fairness testing bar), trains teams, runs the intake process	Doesn't have veto power itself — it recommends into the committee
Legal / privacy	Existing legal function	Reviews data provenance, IP exposure, DPIA-equivalent privacy impact, contract terms with AI vendors	Doesn't own technical risk assessment
Security	Existing security function	Threat-models the system per Module 13 [12-ai-governance/13-ai-security]: prompt injection, data exfiltration, jailbreaks	Doesn't own the business-risk tiering decision
The accountable executive	Financial controls (SOX-style named signer)	A specific person — typically the business owner of the use case, one level below the committee — who signs the deployment decision and owns post-launch monitoring	This is the role most programs skip. Don't skip it.

Why MRM specifically, and why finance got there first: banks have run model risk management for decades because a mispriced trading model or a bad credit model moves real money fast, and regulators (SR 11-7 in the US) require independent validation separate from model development. The AI governance world is re-learning the same lesson banks learned in 2008: *the team that built the model is structurally bad at grading its own homework*, not from bad faith but from proximity bias. Borrowing MRM's core idea — an independent second line of defense that validates before and monitors after deployment — is the single highest-leverage thing a non-financial company can steal from a mature discipline instead of reinventing.

The committee cadence that actually works: monthly for policy and portfolio review; ad hoc, fast-track (48–72 hour SLA) for a single High/Critical-tier launch decision. A committee that only meets monthly for *everything* becomes the bottleneck teams route around — which is how you end up with an unreviewed shadow-IT chatbot in customer support.

3. Model cards and system cards as governance artifacts

A model card is not documentation you write after the fact for compliance theater. It is the **single artifact the committee, MRM, legal, and an auditor a year from now will all read first** — which is why its structure is not optional prose, it's a governance contract.

A complete card contains, at minimum:

Section	Governance reason it exists
Intended use	Defines the boundary the committee approved. Using the system outside this boundary is an unreviewed deployment, full stop.
Out-of-scope / prohibited uses	The explicit "do not use this for X" list — this is what someone points to when a well-meaning team tries to repurpose a customer-support bot for medical advice.
Training/grounding data summary	Provenance for legal (IP, consent, license) and for explaining systematic bias later. For a RAG system this is "what corpus is it grounded in," not "what it was trained on."
Evaluation results	The Measure function's output (Module 15's territory) — accuracy, safety, and fairness metrics <i>with the methodology</i> , not just a headline number. A metric without a method is a marketing claim.
Known limitations & failure modes	What you'd tell a new team member in their first week so they don't get burned by something you already know about.
Human oversight design	Which posture (§4 below) this system requires and why — a Critical-tier card without an oversight section is incomplete by definition.
Change log / version history	Every version bump, prompt change, or re-training event with a date and owner — this is the audit trail (§8).
Governance sign-off	Who approved this, at what tier, on what date — the accountability record.

See `code/model_card_template.md` for a complete, filled-out example (a customer-support RAG assistant), annotated section-by-section with the governance rationale inline. Note that for a compound system built from a foundation model plus retrieval plus your own prompts and guardrails, the more accurate term is a **system card** — the "model" you're describing is really the whole pipeline, and the card should say so rather than pretending there's one artifact called "the model."

4. Human oversight: three postures, and how to avoid the rubber stamp

"Human in the loop" gets used as a synonym for "we're being responsible," which is exactly how it becomes meaningless. There are three distinct postures, and choosing the wrong one for a given risk tier is itself a governance failure.

Posture	What the human does	Trigger condition	Fits which tier
Human-in-the-loop (HITL)	Reviews and approves <i>before</i> the action executes, every time (or every time above a threshold)	Any irreversible or high-stakes individual action: a large refund, a hiring rejection, a triage escalation	High / Critical
Human-on-the-loop (HOTL)	Monitors a stream of autonomous decisions and can intervene, but doesn't approve each one	High-volume, individually low-stakes, reversible actions where 100% review is infeasible	Limited / High
Human-in-command	Sets the operating boundaries and retains a standing kill switch/override authority over the whole system, without touching individual decisions	Systems that must run autonomously at speed (real-time fraud blocking, algorithmic trading-adjacent use cases)	Critical

Designing oversight that isn't a rubber stamp: a HITL gate fails as a control the moment the approver has (a) no real information to evaluate, (b) no time to evaluate it, or (c) no consequence for rubber-stamping. Concretely:

- Show the approver *why* the system proposed this action (the retrieved context, the confidence signal, the alternative it rejected), not just the output — an approve/deny button with no context trains people to click approve.
- Set a review SLA that's actually achievable given the volume. If 500 approvals land on one person's queue per day, you have HOTL with HITL's paperwork and neither's safety. Route volume down (tighten the trigger condition) before you route people faster.
- Sample-audit the approvals themselves. If an approver has a 100% approve rate over 500 decisions, that's a signal the gate isn't doing what you think it's doing — track this as an MRM monitoring metric.
- Make override authority real: a human-in-command posture where the "kill switch" requires filing a ticket that takes three days to action is not human-in-command.

The technical implementation of this policy requirement: this whole section describes a *policy* decision — which posture, what trigger, what the approver sees. Module 16's LangGraph track [\[12-ai-governance/16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md\]](#) covers the mechanics that make it real: `interrupt()` pauses a running graph mid-node and hands control to a human with a specific payload to review, and `Command(resume=...)` lets that human approve, reject, or edit before anything executes — turning "the model did something" into "a named person authorized this at 14:32, and here is exactly what they saw." A governance policy that says "human-in-the-loop required" and an engineering team that has no equivalent of `interrupt()` in their agent loop is a policy with no enforcement mechanism.

5. Third-party and vendor AI risk

Most organizations are not training foundation models. They're calling one (a model API), embedding one (a vendor's "AI-powered" feature bolted onto existing SaaS), or wiring one in via a protocol they didn't write the server for (Module 09 — MCP [\[12-ai-governance/09-mcp\]](#) covers what an MCP server actually is; from a

governance standpoint, an MCP server is a vendor integration that can read your data and take actions, and it deserves the same scrutiny as any other third-party code with tool access).

Standard vendor-security questionnaires ("do you encrypt at rest," "SOC 2 Type II?") don't ask the questions that matter for AI specifically. A vendor-risk assessment for an AI dependency should add:

What to ask the vendor: - What data do you retain from our requests, for how long, and do you train on it? (Get this in writing, not in a sales call — see contract terms below.) - What happens on a model version upgrade — do we get advance notice, a pinned version option, and a changelog of behavior-relevant changes, or does behavior just change under us? - What's your incident disclosure SLA if the model/service is compromised, hallucinates harmfully at scale, or is found to have a safety regression? - Can you provide (or point to) evaluation results, red-team findings, or a model/system card for the specific model version we'd be calling? - What's your uptime/rate-limit behavior under load, and what does your system do if it can't reach its upstream dependency (many AI vendors are themselves wrapping another vendor's model)?

What to require contractually: - A **data-use restriction**: your prompts and outputs are not used to train the vendor's models, unless you've explicitly opted in. - **Version pinning or advance notice** of default-model changes — the single most common way an AI vendor silently breaks your product. - **Audit/inspection rights** proportional to the risk tier of what you're building on top of it. - **Liability and indemnification** language that acknowledges the vendor's model is probabilistic — most standard SaaS contract templates assume deterministic software and don't address this at all. - A **subprocessor/sub-model disclosure** clause — know if your "vendor" is itself a thin wrapper around a foundation-model provider, because your risk actually sits one hop further away than the contract implies.

Tier the review effort the same way you tier use cases: a vendor's embedded "AI writing assistant" in your project-management tool doesn't need the same diligence as replacing your credit-decisioning engine with a vendor model. Reuse the risk-tiering rubric from notes/01 [12-ai-governance/notes/01-governance-fundamentals-and-frameworks.md] §5 — vendor AI risk is use-case risk with an extra column for "how much visibility do we actually have."

6. Change management: why a prompt edit is a deployment

Traditional change management gates code changes. Most organizations' AI systems can change behavior through at least four paths that never touch a pull request review: a system-prompt edit in a config table, an upstream model version bump the vendor pushed, a retrieval corpus update, or a tweaked temperature/sampling parameter. **All four are behavior changes to a governed system, and none of them look like "a change" to a change-management process built for compiled code.**

The governance requirement is simple to state and consistently skipped: **any change that can alter a governed AI system's output distribution requires the same review rigor as the tier it was approved at** — re-run the eval suite, get the sign-off that matches the tier, log the version. A High-tier fraud model doesn't stop being High-tier because the only thing that changed was the prompt.

This is exactly why Module 10's deployment patterns [12-ai-governance/10-ai-architecture] (staged rollout, canary, shadow deployment for AI systems) and Module 16's LangSmith prompt-versioning track [12-ai-governance/16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md] exist as the technical enablers of this policy: a prompt tracked as a versioned, git-adjacent artifact with an eval score attached to each version turns "did anyone review this change" from a question you can't answer into a query you can run. Governance sets the rule (no un-reviewed prompt ships to a High-tier system); engineering practice (prompt versioning, CI-gated evals, canary rollout) is what makes the rule enforceable instead of aspirational.

7. Incident response for AI-specific incidents

An AI incident looks different from a traditional security incident, and a process built only for "system was breached" will miss most of what actually goes wrong:

- A harmful or simply wrong output reached a user and they acted on it (bad medical-adjacent advice, an incorrect financial figure quoted as fact).
- A jailbreak succeeded and the system said something it categorically shouldn't have, publicly.
- A prompt-injection attack (Module 13) caused a tool-using agent to leak data or take an unauthorized action.
- A silent model/vendor version change caused a measurable quality or safety regression in production.
- A bias/fairness finding surfaced post-launch that wasn't caught in pre-launch evaluation.

What a governance-driven response looks like, distinct from a generic incident process:

1. **Triage against the risk tier, not just severity.** A wrong answer from a Minimal-tier internal tool and the same category of wrong answer from a Critical-tier customer-facing system get different response speeds and different sign-off on the fix, even if the immediate blast radius looks similar.
 2. **Contain at the layer that's actually broken** — this might mean rolling back a prompt version, pinning a model version, disabling a tool the agent had access to, or adding a runtime guardrail, and the response owner needs to know which layer they're touching (this is where governance and Module 13's technical controls meet directly).
 3. **Root-cause past "the model did it."** "The model hallucinated" is not a root cause; it's a restatement of the incident. Was the retrieval corpus stale? Was there no eval case covering this input pattern? Was the human-oversight gate bypassed or rubber-stamped? Find the control that should have caught this and didn't.
 4. **Feed the finding back into the risk register.** The point of an incident review is not just fixing this instance — it's adding the failure pattern to the eval suite (so regression is caught automatically next time) and, if the root cause reveals the original risk tier was wrong, re-tiering the use case. An incident that doesn't change the risk register or the eval suite was not actually reviewed, just closed.
 5. **Disclose per the tier and per contract/regulatory obligation.** Some incidents trigger external disclosure requirements — that determination is Module 14 [12-ai-governance/14-ai-compliance]'s territory, but governance owns making sure the question gets asked, not just the engineering fix.
-

8. Documentation and audit trails: governance as a byproduct, not a chore

The instinct is to treat "governance documentation" as a separate paperwork exercise that happens after the real work is done. That instinct produces documentation nobody trusts and nobody updates. The better target: **make the audit trail a byproduct of engineering practices you'd want anyway.**

What a regulator, customer security team, or internal auditor will actually ask for, in practice:

- The AI use-case inventory, with risk tier and approval date for each entry.
- The model/system card for the specific system in question, including its evaluation results.
- Evidence of human oversight design matching the approved posture — and evidence it's *used*, not just documented (approval logs, override rates).
- The change history: every version, what changed, who approved it, what the eval score was before/after.
- The incident log and what changed as a result of each entry.
- Who has access to modify the system (prompts, retrieval sources, model configuration) and evidence that access is reviewed.

Every one of those is a natural output of good engineering practice already covered elsewhere in this track: Module 15's [\[12-ai-governance/15-ai-evals\]](#) eval suite output *is* the evaluation-results evidence. Module 16's LangSmith traces [\[12-ai-governance/16-ai-tech-stack/tracks/langsmith\]](#) *are* the change/version evidence if you tag runs with prompt version and model version. A well-run Module 16 LangGraph [\[12-ai-governance/16-ai-tech-stack/tracks/langgraph\]](#) approval gate's logs *are* the human-oversight evidence. The governance function's job is to make sure these artifacts exist, are retained, and are legible to someone outside engineering — not to invent a parallel paper trail that engineering ignores and auditors don't believe anyway.

9. Where this returns

Idea here	Where it returns
MRM independent validation	Module 15 [12-ai-governance/15-ai-evals] — the eval methodology MRM checks
Human oversight postures	Module 16 LangGraph [12-ai-governance/16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md] §2 — the <code>interrupt()</code> mechanics
Change management for prompts	Module 16 LangSmith [12-ai-governance/16-ai-tech-stack/tracks/langsmith/notes/02-evaluation-and-operations.md] §7 — prompt versioning
Vendor/MCP risk	Module 09 — MCP [12-ai-governance/09-mcp] — what you're actually trusting when you add a server
Incident containment layer	Module 13 — AI Security [12-ai-governance/13-ai-security] — the technical controls an incident response touches
Disclosure obligations	Module 14 — AI Compliance [12-ai-governance/14-ai-compliance] — which incidents trigger legal reporting